



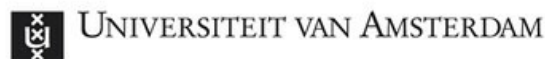
Print test 42727 15402

Introduction to python (Universiteit van Amsterdam)



messages.pdf_cover_qr_code_label

Test Resit - Introduction to Python - 22/23



Session period: 12 July 2023 09:00 - 12:00

Duration: 120 minutes

Instruction

Economics and Business
Academic Year 2022-2023

Exam: Introduction to Python 6013B0470Y

Date and time of the exam: 12-7-2023, 9:00 - 11:00

Duration of the exam: 2 hours

IDENTIFICATION:

During the examination you must be able to prove your identity by means of a valid student ID card, valid passport, ID card or driving license.

EXAM RULES:

The **Rules and Guidelines of the Examinations Board 2022-2023** apply to this exam; make sure you adhere to all elements thereof. Violation of the **Rules and Guidelines** does not only invalidate your exam, but could also constitute fraud. Students who are in violation or suspected of fraud will be referred to the Board of Examinations.

Mobile telephones or other **electronic equipment** should be switched off and stored in a locker or if no lockers are available in a bag or briefcase next to the examination table. The bag or briefcase should be closed at all times.

During the exam you are **not allowed** to go to the toilet (unless, by way of exception, this is approved by the Examiner or Head invigilator).

RESOURCES DURING THE EXAM

You may only have a **pen and a ruler** on your table and, in case of a *handwritten* multiple choice exam, a pencil and an eraser, unless supplementary aids are indicated on this cover page of the examination.

The use of **graphing and/or programmable calculators** during an examination is never permitted, unless it is explicitly stated on this cover page of the examination that their use is permitted.

If scrap paper is used, it is **mandatory** to write your name and student number on **every sheet of paper** before use.

Reference works **that contain annotations are not allowed**.

Supplementary aids specifically for this exam:

- Simple Calculator
- One double-sided A4 sheet of personal notes, handwritten or printed.

Declaration of authenticity

By submitting answers in this exam, you affirm that those answers are yours, and yours alone. You affirm that you have written all texts and decided upon the answers without any outside assistance or the use of inadmissible sources or tools. You affirm that you were using your own personal UvA credentials to login on the exam computer and/or identified yourself during the exam and to submit your work. You are aware of all stipulations and procedures in the **Rules and Guidelines of the Examinations Board 2022-2023**, and that those will apply to this exam.

Question 1 – Multiple choice – Question-ID: 357615 (1 point)

Suppose you have a list called **x** containing tuples, where

```
x = [(1, 2), (3, 4), (5, 6), (7, 8)]
```

Which of the following lines of code will print `True`?

- A** `print(x[0][1] * x[-3][0] == x[2][1])`
 - B** `print(x[1][0] + x[-2][0] == x[-1][1])`
 - C** `print(x[-1][0] - x[-2][0] == x[0][1])`
 - D** All lines of code will print `True`.
-

Question 2 – Multiple choice – Question-ID: 357607 (1 point)

Executing the following program will produce an error. Why?

```
def power(num, factor):  
    result = num ** factor  
    return 'The result of {} ** {} is: {}'.format(num, factor, result)  
print(result)
```

- A** The function called **power** should not return the string `'The result of {} ** {} is: {}'.format(num, factor, result)`, but the value of **result**.
 - B** `**` is not a valid Python operator.
 - C** The syntax used in the return statement to format the string is incorrect.
 - D** **Result** is a local name that cannot be referenced outside of the function **power**.
-

Question 3 – Multiple choice – Question-ID: 355235 (1 point)

What will be printed by the following code segment?

```
def main(x):  
    y = list()  
    for i in x:  
        if type(i) == type(x):  
            y.append(i)  
    return y  
  
result = main(['a', 'None', [], {}, '123', 0.05, 111])  
print(result)
```

- A** `['a', 'None', '123']`
 - B** `[[]]`
 - C** `[]`
 - D** `[None]`
-

Question 4 – Multiple choice – Question-ID: 357677 (1 point)

Suppose you have the following function, which takes two integers as arguments:

```
def main(x, y=11):  
    table = []  
    i = 1  
    while i < y:  
        table.append(f'{x} * {i} = {x*i}')  
        i += 1  
    return table
```

What will be printed by the following line of code:

```
print(main(1))
```

A This code produces an error as there is a missing argument when calling **main**.

B

```
['1 * 1 = 1',  
 '1 * 2 = 2',  
 '1 * 3 = 3',  
 '1 * 4 = 4',  
 '1 * 5 = 5',  
 '1 * 6 = 6',  
 '1 * 7 = 7',  
 '1 * 8 = 8',  
 '1 * 9 = 9',  
 '1 * 10 = 10']
```

C The code produces a syntax error because of the line:

```
table.append(f'{x} * {i} = {x*i}')
```

D

```
['1 * 1 = 1',  
 '1 * 2 = 2',  
 '1 * 3 = 3',  
 '1 * 4 = 4',  
 '1 * 5 = 5',  
 '1 * 6 = 6',  
 '1 * 7 = 7',  
 '1 * 8 = 8',  
 '1 * 9 = 9',  
 '1 * 10 = 10',  
 '1 * 11 = 11']
```

Question 5 – Multiple choice – Question-ID: 357812 (1 point)

Suppose you have a list called **x**, where:

```
x = [1, 1, 2, 2, 3, 3, 4, 4, 5, 5]
```

Which of the following code segments will print the following list?

```
[5, 4, 3, 2, 1]
```

A

```
y= list(set(x))  
print(sorted(y, reverse = True))
```

B

```
print(x[::-2])
```

C Both lines of code will print that output.

D Neither line of code will print that output.

Question 6 – Multiple choice – Question-ID: 357749 (1 point)

You need to write a function called **main** which accepts a flexible number of keyword arguments. The function should compute and return the sum of the values of the arguments.

For example, if you call the function as:

```
main(apple=1.99, banana=0.99, orange=1.49)
```

it should return:

```
4.47
```

Which of the following functions will return the intended value?

- A**

```
def main(**kwargs):  
    total = 0  
    for item, price in kwargs.items():  
        total += item  
    return total
```
 - B**

```
def main(**kwargs):  
    total = 0  
    for price in kwargs.values():  
        total += price  
    return total
```
 - C**

```
def main(**kwargs):  
    total = 0  
    for item, price in kwargs.keys():  
        total += price  
    return total
```
 - D** All functions will return the intended value.
-

Question 7 – Multiple choice – Question-ID: 360558 (1 point)

Assume you already have a list of strings called **words**.

You need to create a dictionary called **word_lengths**, where the keys are the strings in the list **words** and the values are their lengths.

Which of the following code segments will work as intended?

- A**

```
word_lengths = {length: word for word, length in enumerate(words)}
```
 - B**

```
word_lengths = {len: word for word in words}
```
 - C**

```
word_lengths = {word: len(word) for word in words}
```
 - D**

```
word_lengths = {word: len for word, len in words}
```
-

Question 8 – Multiple choice – Question-ID: 360559 (1 point)

You are given two lists, called **students** and **grades**:

- The **students** list contains strings representing the names of students.
- The **grades** list contains integers representing the students' corresponding grade levels.

You need to create a dictionary called **grade_counts** that counts the number of students in each grade level. That is, the keys of the dictionary are the grade levels and the values are the number of students in that grade.

For example, if you have:

```
students = ['A', 'B', 'C', 'D', 'E', 'F']
```

```
grades = [9, 10, 9, 11, 10, 9]
```

then the created dictionary **grade_counts** should be:

```
{9: 3, 10: 2, 11: 1}
```

Which of the following code segments will work as intended?

- A**

```
grade_counts = {}  
for grade in grades:  
    if grade not in grade_counts:  
        grade_counts[grade] = 0  
    grade_counts[grade] += 1
```
- B**

```
grade_counts = {}  
for student, grade in zip(students, grades):  
    if grade in grade_counts:  
        grade_counts[grade].append(student)  
    else:  
        grade_counts[grade] = [student]
```
- C** Both of the given options will work as intended.
- D** None of the given options will work as intended.
-

Question 9 – Multiple choice – Question-ID: 361882 (1 point)

You have a dictionary that represents the scores of students in a math competition:

```
math_scores = {'John': 85, 'Sarah': 92, 'Michael': 78, 'Emma': 90, 'David': 82}
```

You want to find and print the names of the students who scored above the average score.

Which of the following code segments will work as intended?

- A**

```
average_score = sum(math_scores.values())/len(math_scores)  
for student in math_scores.keys():  
    if math_scores[student] > average_score:  
        print(student)
```
- B**

```
for student in math_scores:  
    if math_scores[student] > (sum(math_scores.values())/len(math_scores)):  
        print(student)
```
- C** Both code segments will work as intended.
- D** None of the code segments will work as intended.
-

Question 10 – Multiple choice – Question-ID: 361844 (1 point)

What is the output of the following code segment?

```
x = 'Hello, World!'
print([char for char in x if char.islower()])
```

Hint:
The `islower()` string method returns `True`, if and only if all the characters in that string are lower case letters, in all other cases it returns `False`.

- A** ['e', 'l', 'l', 'o', 'o', 'r', 'l', 'd']
 - B** ['e', 'l', 'l', 'o', ',', 'o', 'r', 'l', 'd', '!']
 - C** ['h', 'e', 'l', 'l', 'o', 'w', 'o', 'r', 'l', 'd']
 - D** ['H', 'e', 'l', 'l', 'o', 'W', 'o', 'r', 'l', 'd']
-

Question 11 – Multiple choice – Question-ID: 361839 (1 point)

What will be the output of the following code segment?

```
list1 = [5, 10, 15, 20]
list2 = [2, 4, 6, 8]
result = []

for a, b in zip(list1, list2):
    result.append(a // b)

print(result)
```

- A** [1, 2, 3, 4]
 - B** [2, 2, 2, 2]
 - C** [6, 12, 18, 24]
 - D** [2.5, 2.5, 2.5, 2.5]
-

Question 12 – Multiple choice – Question-ID: 361835 (1 point)

Suppose you have a list called **numbers** = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

Which of the following lines of code **will not** create a list with the name **subset**, and the value [10, 8, 6, 4, 2]

- A** subset = numbers[::2][::-1]
 - B** subset = numbers[-1::-2]
 - C** subset = numbers[::-1][::2]
 - D** subset = numbers[:::-2]
-

Question 13 – Multiple choice – Question-ID: 361941 (1 point)

You have a string variable called **sentence**:

```
sentence = 'I bought a book about programming. The book has a lot of exercises. I enjoy reading a good book.'
```

You want to replace **only** the second occurrence of "book" with "novel" in the sentence and save this as **new_sentence**.

So:

```
print(new_sentence)
```

should print:

```
I bought a book about programming. The novel has a lot of exercises. I enjoy reading a good book.
```

Which of the following code segments will work as intended?

Hint:

The string `find()` method takes three arguments: value, start and end. The first argument is required and is the value to search for. The second and third argument are optional, they are indices specifying where to start and end the search, respectively.

- A**

```
index = sentence.find('book')
index = sentence.find('book', index + 1)
new_sentence = sentence[:index] + 'novel' + sentence[index + 4:]
```
 - B**

```
new_sentence = sentence.replace("book", "novel", 2)
new_sentence = new_sentence.replace("novel", "book", 1)
```
 - C** Both options will work as intended.
 - D** None of the options will work as intended.
-

Question 14 – Multiple choice – Question-ID: 361960 (1 point)

You have the following objects loaded in Python:

```
fruits = ["apple", "banana", "orange"]
count = 3
```

You want to print the following sentence:

```
I have 3 fruits: apple, banana, and orange.
```

Which of the following code fragments will achieve the desired output?

- A**

```
message = f"I have {count} fruits: {'', '.join(fruits)}.'"
print(message)
```
 - B**

```
message = f"I have {count} fruits: {fruits[0]}, {fruits[1]}, and {fruits[2]}.'"
print(message)
```
 - C** Both code fragments will achieve the desired output.
 - D** Neither of the code fragments will achieve the desired output.
-

Question 15 – Multiple choice – Question-ID: 361942 (1 point)

A company wants to extract the components of email addresses. An email address consists of the local part, which comes before the "@" symbol and a domain name, which comes after the "@" symbol.

You need to create a function called **main()** which accepts a flexible number of strings representing email addresses and returns a DataFrame. Your DataFrame should have a column called **'local'** which contains the local parts and a column called **'domain'** which contains the domain names of the email addresses that were passed as arguments.

For example, if you execute the following command:

```
print(main('example@example.com', 'student@uva.nl', 'email@address.com'))
```

the output should be:

	local	domain
0	example	example.com
1	student	uva.nl
2	email	address.com

Which of the following code segments will return this dataframe as intended?

You may assume that pandas is already imported as **pd**.

- A**

```
def main(*args):  
    local = []  
    domain = []  
    for email in args:  
        local.append(email.split("@")[0])  
        domain.append(email.split("@")[1])  
    return pd.DataFrame({local: domain for local, domain in zip(local, domain)})
```
- B**

```
def main(*args):  
    data = {}  
    for email in args:  
        data['local'] = email.split("@")[0]  
        data['domain'] = email.split("@")[1]  
    return pd.DataFrame(data)
```
- C**

```
def main(args):  
    return pd.DataFrame({  
        'local': [email.split("@")[1] for email in args],  
        'data' : [email.split("@")[0] for email in args]})
```
- D**

```
def main(*args):  
    data = {}  
    data['local'] = pd.Series([email.split("@")[0] for email in args])  
    data['domain'] = pd.Series([email.split("@")[1] for email in args])  
    return pd.DataFrame(data)
```
-

Question 16 – Multiple choice – Question-ID: 358033 (1 point)

You have the following definition of a class called **Employee**:

```
class Employee:
    def __init__(self, name, role, gender = 'not specified'):
        self.name = name
        self.role = role
        self.gender = gender
```

What will be the output if we execute the following lines of code?

```
employee1 = Employee('John Smith', 'HR representative')
print(employee1.name == 'John Smith')
print(employee1.gender == 'not specified')
```

- A** True
True
 - B** False
True
 - C** False
False
 - D** True
False
-

Question 17 – Multiple choice – Question-ID: 358040 (1 point)

Which other code fragment will print the same output as the following?

```
from datetime import datetime, timedelta
print((datetime.strptime('2023/1/10', '%Y/%m/%d') - datetime(2023, 1, 1)).days + 1)
```

- A** from datetime import datetime, timedelta
print(timedelta(datetime(2023, 1, 10) - datetime(2023, 1, 1)).days)
 - B** from datetime import datetime, timedelta
print(timedelta(datetime(2023, 1, 10) - datetime(2023, 1, 1)).days + 1)
 - C** from datetime import datetime, timedelta
print((datetime(2023, 1, 10) - datetime.strptime('2023-1-1', '%Y-%m-%d')).days)
 - D** from datetime import datetime, timedelta
print((datetime(2023, 1, 11) - datetime(2023, 1, 1)).days)
-

Question 18 – Multiple choice – Question-ID: 361961 (1 point)

Suppose you have the following DataFrame called **df**:

	Player	Age	Height
0	John	25	180
1	Jane	30	165
2	Alex	27	175

You may assume that pandas is already imported as **pd**.

Which of the following lines of code will give an error?

- A** `print(df[['Player', 'Age']])`
 - B** `print(df.loc[df['Height'] > 170, ['Player', 'Age']])`
 - C** `print(df.loc[:, 'Age'])`
 - D** `print(df[1, 'Age'])`
-

Question 19 – Multiple choice – Question-ID: 358031 (1 point)

You have the following definition for a class called **Animals**:

```
class Animals:
    def __init__(self, name, sound, legs=None, fin=None):
        self.name = name
        self.sound = sound
        self.legs = legs
        self.fin = fin
    # missing lines
    # ...
```

You need to add two methods to the class:

1. The **walk()** method, which returns the '**sound**' attribute an Animal object has if the Animal object has '**legs**'. If the Animal object does not, return 'Cannot walk!'.
2. The **swim()** method, which returns the '**sound**' attribute an Animal object has if the Animal object has '**fin**'. If the Animal object does not, return 'Cannot swim!'.

For example, if we execute the following commands:

```
cat = Animals('kitty', legs=True, sound='Meow!')
fish = Animals('nemo', fin=True, sound='Blob!')
print(cat.walk())
print(cat.swim())
print(fish.swim())
print(fish.walk())
```

the output should be:

```
Meow!
Cannot swim!
Blob!
Cannot walk!
```

Which of the following code segments should you place on the missing lines in the class definition to achieve what you want?

A

```
def walk(self):
    return self.sound if self.legs else 'Cannot walk!'
def swim(self):
    return self.sound if self.fin else 'Cannot swim!'
```

B

```
def walk(self):
    if self.legs == True:
        return self.sound
    else:
        return 'Cannot walk!'
def swim(self):
    if self.fin == True:
        return self.sound
    else:
        return 'Cannot swim!'
```

C

```
def walk():
    return sound if legs else 'Cannot walk!'
def swim():
    return sound if fin else 'Cannot swim!'
```

D

```
def walk():
    if legs == True:
        return sound
    else:
        return 'Cannot walk!'
def swim():
    if fin == True:
        return sound
    else:
        return 'Cannot swim!'
```

Question 20 – Multiple choice – Question-ID: 357851 (1 point)

You have the following definition for a class named **Movie**.

```
class Movie:
    def __init__(self, title, director, reviews = None):
        self.title = title
        self.director = director
        self.reviews = reviews if reviews else []
    # missing lines
    # ...
```

You need to add three methods to the class:

1. The **add_review()** method, which adds review scores to the '**reviews**' attribute,
2. The **rating()** method, which computes and returns the average review score of all scores stored in the '**reviews**' attribute. If there are no reviews, the method should return 0.
3. The **compare()** method, which compares the average review scores of two Movie objects and return the title of the Movie object with the higher average review score, i.e., the name of the movie you should watch. In case both Movie objects have the same average review score, the method should return 'Either'.

For example, if we execute the following commands:

```
movie1 = Movie('In Bruges', 'Martin McDonagh', [5, 3, 5, 4])
movie2 = Movie('Parasite', 'Bong Joon Ho')
print(movie1.compare(movie2))
```

the output should be:

In Bruges

Which of the following code segments should you place on the missing lines in the class definition to achieve what you want?

- A**
- ```
def add_review(self, score):
 self.reviews += score
def rating(self):
 return sum(self.reviews)/len(self.reviews) if self.reviews else 0
def compare(self, other):
 for first, second in [(self, other), (other, self)]:
 return first.title if first.rating() > second.rating() else 'Either'
```
- B**
- ```
def add_review(score):
    reviews.append(score)
def rating():
    if reviews:
        return sum(reviews)/len(reviews)
    else:
        return 0
def compare(self, other):
    if self.rating > other.rating:
        return self.title
    elif self.rating < other.rating:
        return other.title
    else:
        return 'Either'
```
- C** Both code segments will work as intended.
- D** Neither code segments will work as intended.
-

Question 21 – Multiple choice – Question-ID: 358072 (1 point)

You need to write a function called **main**, which accepts a string as an argument. The string represents a date as DD-MM-YYYY.

The function should return a list containing 10 strings representing dates, all in that same format.

The first element of the list should be the original input date.

Each further element of the list should be a string representing the date that is exactly one week after the preceding element of the list.

Take a look at the following code segment:

```
from datetime import datetime, timedelta

def main(string):
    date = datetime.strptime(string, '%d-%m-%Y')
    str_list = [date]
    for i in range(1, 10):
        str_list.append((str_list[i-1] + timedelta(weeks=1)).strftime('%d-%m-%Y'))
    return str_list
```

Will this code segment work as intended? If not, then why not?

- A** This code will not work as intended, because there will be 11 strings in the list instead of 10.
 - B** The code will not work because the two methods **strptime()** and **strftime()** are mixed up.
 - C** This code will not work because **str_list[i-1]** is a string. Timedelta objects can only be added to datetime objects, and cannot be added to strings.
 - D** This code segment will work as intended.
-

Question 22 – Multiple choice – Question-ID: 358968 (1 point)

Suppose you have the following DataFrame called **df**:

	Day	Month	Year
0	1	5	2023
1	8	5	2023
2	15	5	2023
3	22	5	2023
4	29	5	2023

You need to create a new column called **Date** that contains strings representing the dates as constructed from the **Day**, **Month** and **Year** columns. That is, finally the DataFrame should look like this:

	Day	Month	Year	Date
0	1	5	2023	01-05-2023
1	8	5	2023	08-05-2023
2	15	5	2023	15-05-2023
3	22	5	2023	22-05-2023
4	29	5	2023	29-05-2023

Which of the following code segments will achieve what you want? You may assume that the pandas is already imported as **pd**.

- A**

```
from datetime import datetime
df['Date'] = pd.Series([datetime(y, m, d).strftime('%d-%m-%Y') for d, m, y in
zip(df['Day'], df['Month'], df['Year'])])
```
 - B**

```
from datetime import datetime
df['Date'] = pd.Series([datetime.strptime(d, m, y, '%d-%m-%Y') for d, m, y in
zip(df['Day'], df['Month'], df['Year'])])
```
 - C**

```
from datetime import datetime
df['Date'] = pd.Series([datetime(d, m, y).strftime('%d-%m-%Y') for d, m, y in
zip(df['Day'], df['Month'], df['Year'])])
```
 - D**

```
from datetime import datetime
df['Date'] = pd.Series([datetime.strptime(date, '%d-%m-%Y') for date in
zip(df['Day'], df['Month'], df['Year'])])
```
-

Question 23 – Multiple choice – Question-ID: 361969 (1 point)

Consider the following pandas DataFrame called **df**:

	Name	Age	Salary
0	John	25	50000
1	Jane	30	65000
2	Alex	35	80000
3	Lisa	40	70000
4	Mark	28	55000

You need to perform a series of operations on this DataFrame:

- Operation 1: Create a new column called **Above_average**, which contains `True` if the employee's salary is above the average salary of all the employees, otherwise `False`.
- Operation 2: Create a new column called **Salary_difference**, which represents how much higher an employee's salary is than the average salary.
- Operation 3: Print the segment of the adapted DataFrame that contains all the columns, but only the rows in which the employees earn more than the average salary.

That is, the final result printed should be:

	Name	Age	Salary	Above_average	Salary_difference
1	Jane	30	65000	True	1000
2	Alex	35	80000	True	16000
3	Lisa	40	70000	True	6000

Which of the following blocks of code will perform these operations as intended? You may assume the pandas module is already imported as **pd**.

- A**

```
mean_salary = round(df['Salary'].mean())
df['Above_average'] = pd.Series([True if salary > mean_salary else False for salary
in df['Salary']])
df['Salary_difference'] = df['Salary'] - mean_salary
print(df[df['Above_average']==True])
```
- B** None of the blocks of code will work as intended.
- C** Both blocks of code will work as intended.
- D**

```
mean_salary = round(df['Salary'].mean())
df['Above_average'] = df['Salary'].map(lambda x: True if x > mean_salary else
False)
df['Salary_difference'] = df['Salary'].map(lambda x: x - mean_salary)
print(df[df['Salary_difference']>0])
```
-

Question 24 – Multiple choice – Question-ID: 361968 (1 point)

What will be the output of the following lines of code?

```
import pandas as pd

df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})
df['C'] = df['A'].map(lambda x: x + df['B'].sum())

print(df)
```

A

	A	B	C
0	1	4	18
1	2	5	21
2	3	6	24

B

	A	B	C
0	1	4	12
1	2	5	15
2	3	6	18

C

	A	B	C
0	1	4	9
1	2	5	12
2	3	6	15

D

	A	B	C
0	1	4	16
1	2	5	17
2	3	6	18

End of test Resit - Introduction to Python - 22/23